FN6811: Analysis of SSE Composite Index Return Dynamics and Volatility

Wei Yuan Oh, G2500601C

1. Introduction and Data Preparation

1.1 Assignment Context

This report supports the company's initiative to invest in the Shanghai Pilot Free Trade Zone (SPFTZ). We aim to construct a portfolio replicating the Shanghai Stock Exchange (SSE) Composite Index. The objective is to model the return dynamics and volatility of the SSE using a rigorous **Joint Estimation** methodology to select the optimal model.

1.2 Data Ingestion and Transformation

We analyze the daily closing prices from January 2, 2008, to October 15, 2025. We calculate **daily log returns** (R_t):

$$R_t = 100 \times \ln\left(\frac{PRICE_t}{PRICE_{t-1}}\right)$$

```
filename <- "SSE_Jan2008_Oct2025.xlsx"

raw_data <- suppressWarnings(
  read_excel(filename, col_types = c("text", "text"))
)
names(raw_data) <- c("Date", "Price")

df <- raw_data %>%
  mutate(
    Price = as.numeric(Price),
    Date_clean = str_remove(Date, "\\s.*$"),

    Date_Parsed = case_when(

      str_detect(Date_clean, "/") ~ mdy(Date_clean),

      str_detect(Date_clean, "-") ~ ydm(Date_clean),

      str_detect(Date_clean, "^[0-9]+$") ~ {
```

```r
        d_temp <- as.Date(as.numeric(Date_clean), origin = "1899-12-30")
        d_str <- format(d_temp, "%Y-%m-%d")
        ydm(d_str)
      },

      TRUE ~ as.Date(NA)
    )
  ) %>%
  filter(!is.na(Date_Parsed)) %>%
  filter(Date_Parsed <= as.Date("2025-10-31")) %>%
  arrange(Date_Parsed) %>%
  distinct(Date_Parsed, .keep_all = TRUE)

df <- df %>%
  mutate(
    ln_price = log(Price),
    Return = 100 * (ln_price - lag(ln_price))
  ) %>%
  drop_na()

sse_xts <- xts(df$Return, order.by = df$Date_Parsed)
colnames(sse_xts) <- "SSE_Return"

# --- Verification ---
cat(sprintf("Rows processed: %d\n", nrow(df)))
```

```
Rows processed: 4321
```

```r
cat(sprintf("Date Range: %s to %s\n", min(index(sse_xts)), max(index(sse_xts))))
```

```
Date Range: 2008-01-03 to 2025-10-15
```

```r
head(sse_xts)
```

```
           SSE_Return
2008-01-03  0.8883560
2008-01-04  0.7809855
2008-01-07  0.5908016
2008-01-08 -0.1263466
2008-01-09  0.9107151
2008-01-10  0.3806346
```

```r
tail(sse_xts)
```

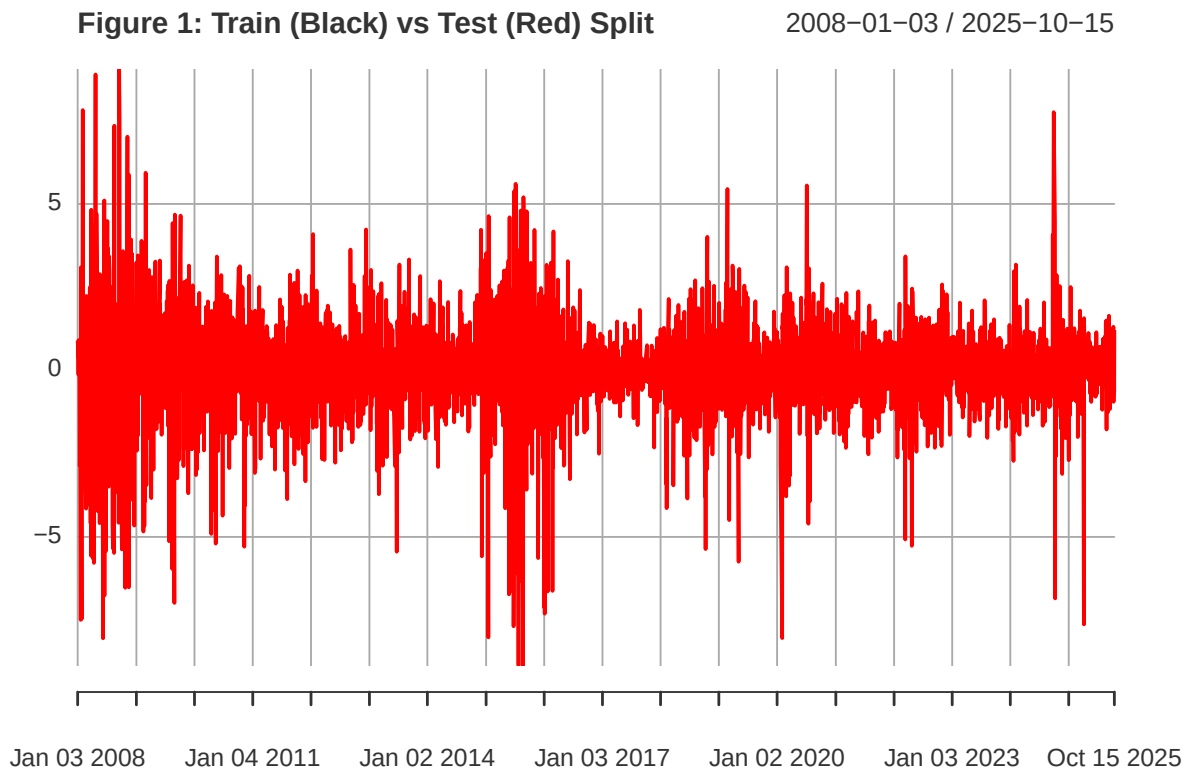
```
           SSE_Return
2025-09-30  0.5228983
2025-10-09  1.3097702
2025-10-10 -0.9434370
2025-10-13 -0.1934110
2025-10-14 -0.6259426
2025-10-15  1.2081243
```

1.3 Train-Test Split (70:30)

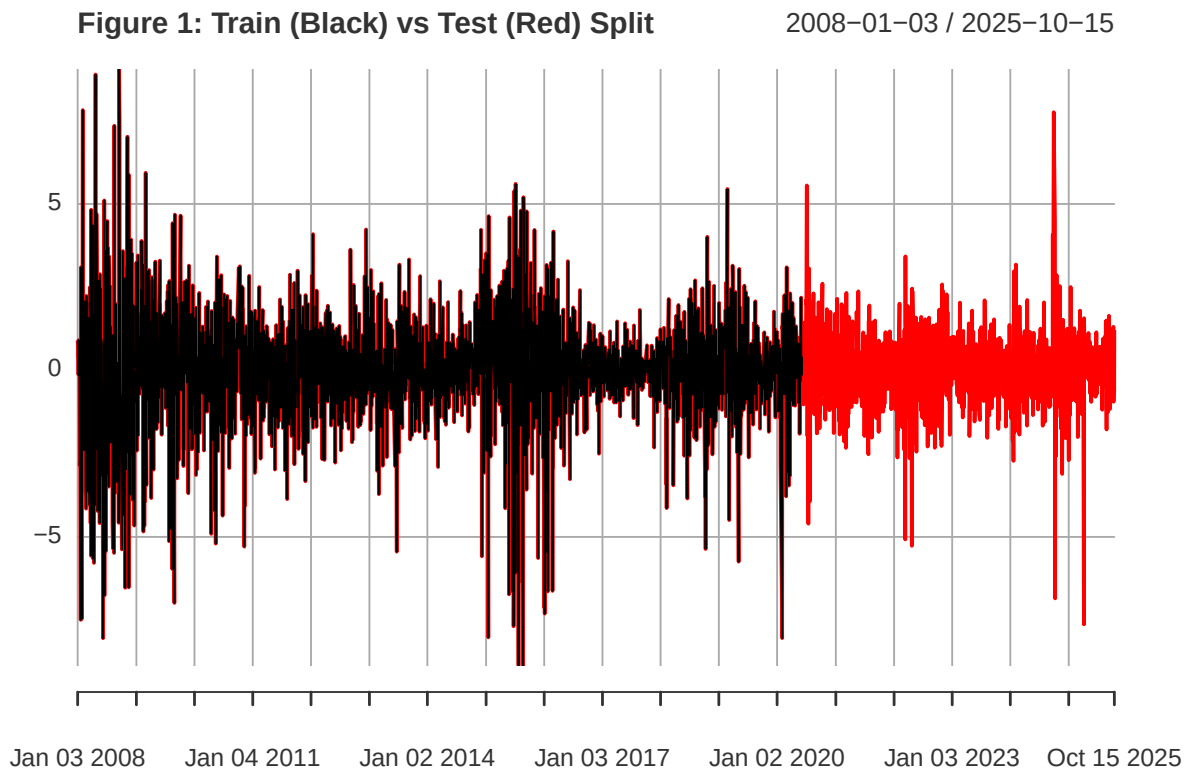
To ensure our model is robust for investment decisions, we strictly separate the data into:

1. **Training Set (70%)**: Used for Identification, Joint Estimation, and Model Selection.
2. **Testing Set (30%)**: Used exclusively for Forecasting and Out-of-Sample Evaluation.

```r
# Determine split index
n_total <- length(sse_xts)
n_train <- floor(0.70 * n_total) # 70% for training
n_test  <- n_total - n_train     # 30% for testing (holdout)

# Create Subsets
sse_train <- sse_xts[1:n_train, ]
sse_test  <- sse_xts[(n_train + 1):n_total, ]

# Visualize Split
par(mar = c(3, 4, 3, 1))
plot(
  sse_xts,
  main          = "Figure 1: Train (Black) vs Test (Red) Split",
  major.ticks   = "years",
  grid.ticks.on = "years",
  col           = "red",
  yaxis.right   = FALSE
)
```




```
lines(sse_train, col = "black")
legend(
  "topright",
  legend = c("Training Set", "Testing Set"),
  col    = c("black", "red"),
  lty    = 1,
  cex    = 0.8,
  bty    = "n"
)
```




```r
cat(sprintf(
  "Training Data: %d observations\nTesting Data: %d observations",
  length(sse_train), length(sse_test)
))
```

```
Training Data: 3024 observations
Testing Data: 1297 observations
```

2. Statistical Properties (Training Set)

We perform all statistical checks on the **Training Set** only.

2.1 Descriptive Statistics & Normality

```r
# Calculate Stats on TRAIN
stats <- table.Stats(sse_train)
kable(stats, caption = "Descriptive Statistics (Training Set)")
```

Table 1: Descriptive Statistics (Training Set)

	SSE_Return
Observations	3024.0000
NAs	0.0000
Minimum	-8.8729
Quartile 1	-0.6574
Median	0.0554
Arithmetic Mean	-0.0193
Geometric Mean	NaN
Quartile 3	0.7181
Maximum	9.0343
SE Mean	0.0289
LCL Mean (0.95)	-0.0761
UCL Mean (0.95)	0.0374
Variance	2.5344
Stdev	1.5920
Skewness	-0.5895
Kurtosis	5.0208

```r
# Kurtosis Check
kurt_val <- kurtosis(sse_train)
cat("Excess Kurtosis:", kurt_val, "\n")
```

```
Excess Kurtosis: 5.020757
```

Decision: The Training data exhibits significant Excess Kurtosis (5.02). We reject Normality and will use the **Student-t distribution** in our GARCH models, treating inference as robust/QMLE rather than relying on strict Gaussian MLE assumptions.

2.2 Stationarity Test (ADF)

```r
adf_res <- adf.test(sse_train)
print(adf_res)
```

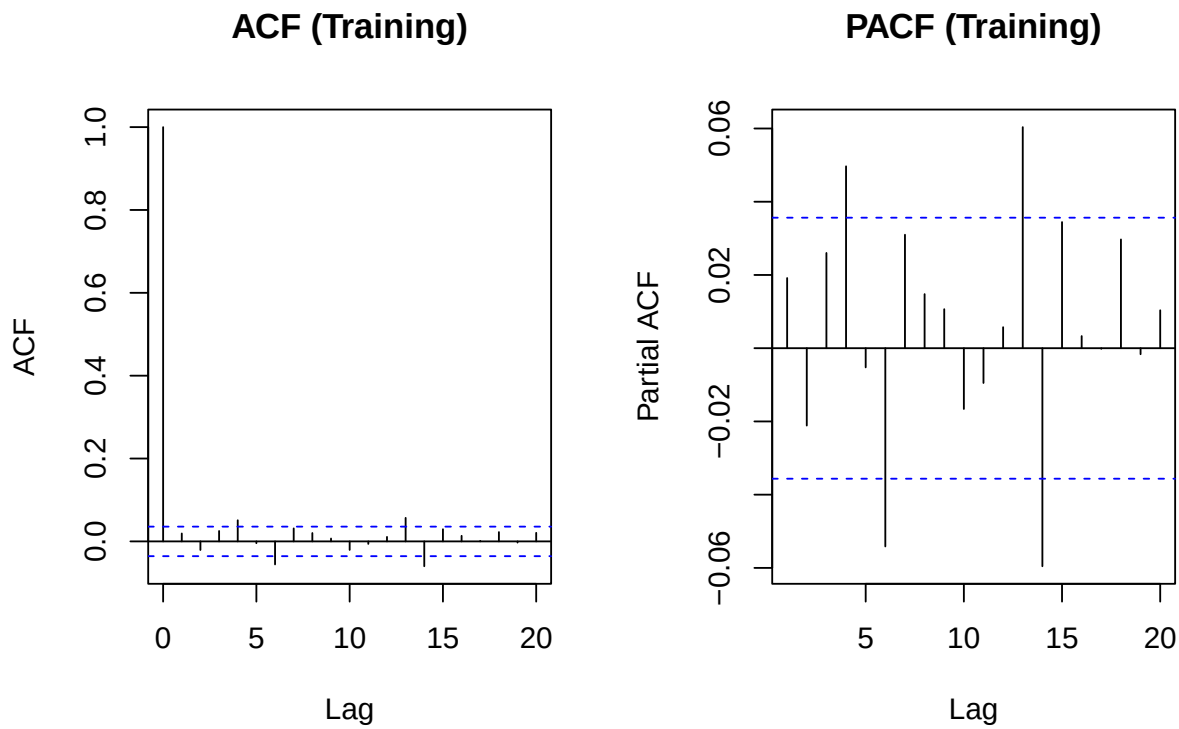
```
    Augmented Dickey-Fuller Test

data:  sse_train
Dickey-Fuller = -13.685, Lag order = 14, p-value = 0.01
alternative hypothesis: stationary
```

Conclusion: The p-value is < 0.05. The Training series is **stationary**, so no further differencing is required.

3. Mean Equation Identification

We inspect the correlograms of the Training set to identify plausible Mean Equation candidates.

```
par(mfrow = c(1, 2))
acf(sse_train,  main = "ACF (Training)",  lag.max = 20)
pacf(sse_train, main = "PACF (Training)", lag.max = 20)
```




```
par(mfrow = c(1, 1))
```

Interpretation:

- The significant spike at PACF lag 1 suggests an AR(1).
- The ACF decay pattern suggests possible ARMA structure.
- **Candidates:** We will test a family of mean models: **AR(1), MA(1), ARMA(1,1)** paired with GARCH-type volatility components.

4. Formal Test for ARCH Effects (Baseline Mean Model)

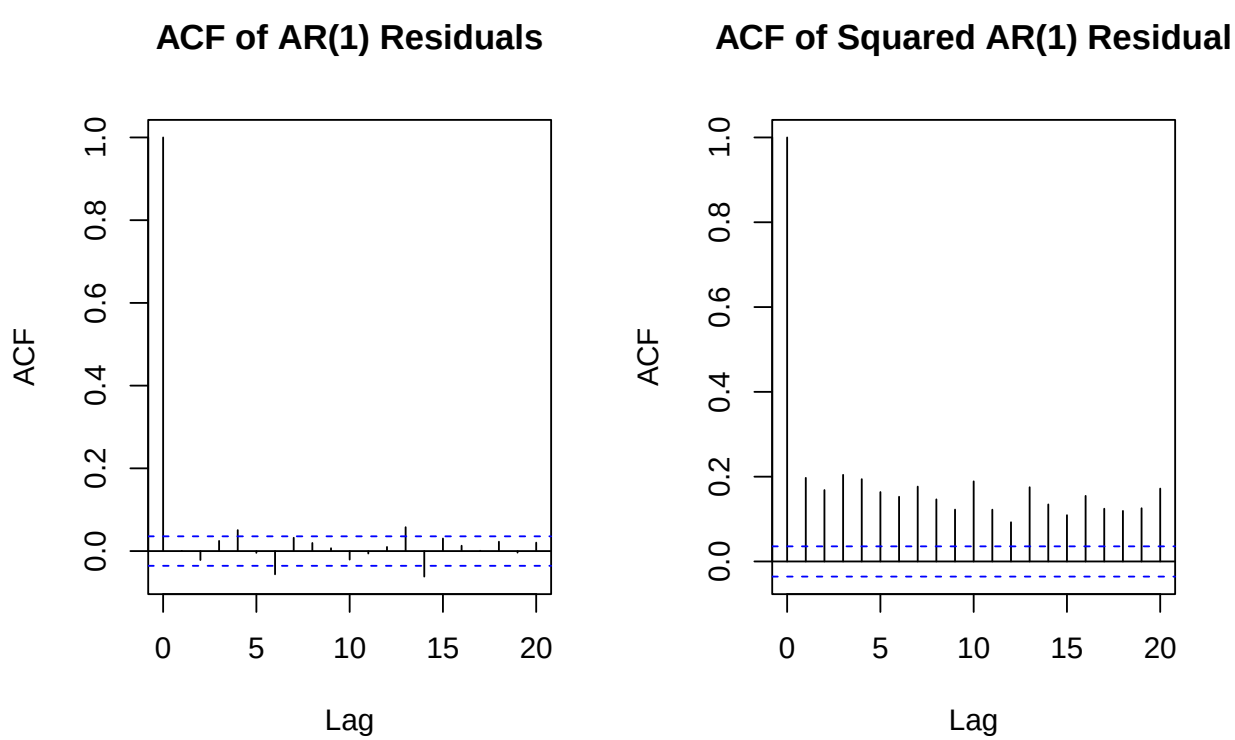
Before proceeding to GARCH-type volatility modelling, we formally confirm that **volatility clustering (ARCH effects)** exists in the residuals of a simple mean equation.

We fit a baseline **AR(1)** model and apply the **ARCH Lagrange Multiplier (LM) Test** and residual correlograms to its residuals.

```
mean_model_baseline <- arima(as.numeric(sse_train), order = c(1, 0, 0))


resids_baseline <- residuals(mean_model_baseline)


# H0: No ARCH effects (Homoskedasticity)
# H1: ARCH effects present (Heteroskedasticity)
arch_test_res <- ArchTest(resids_baseline, lags = 12)
print(arch_test_res)
```

```
    ARCH LM-test; Null hypothesis: no ARCH effects

data:  resids_baseline
Chi-squared = 362.97, df = 12, p-value < 2.2e-16
```

```
par(mfrow = c(1, 2))
acf(resids_baseline,   main = "ACF of AR(1) Residuals",        lag.max = 20)
acf(resids_baseline^2, main = "ACF of Squared AR(1) Residual", lag.max = 20)
```



```r
par(mfrow = c(1, 1))
```

Conclusion: The ARCH-LM test yields a p-value < 0.05, and the correlogram of squared residuals shows significant autocorrelation. We therefore **reject the null of no ARCH effects**:

- There is statistically significant **volatility clustering** in the residuals of the AR(1) model.
- **Implication:** A constant-variance AR/ARMA model is inadequate; we proceed to **GARCH-type models** for the conditional variance.

5. Joint Model Selection

Having confirmed the need for GARCH, we now test **Joint Mean–Variance specifications**. To remain within the course scope, we restrict volatility models to:

- **GARCH(1,1)** (rugarch: `sGARCH`)
- **GJR-GARCH(1,1)** (rugarch: `gjrGARCH`)
- **EGARCH(1,1)** (rugarch: `eGARCH`)
- **GARCH-in-Mean** (volatility enters the mean equation; risk–return trade-off)

To strengthen identification, we run a compact grid:

- Mean equations: **AR(1), MA(1), ARMA(1,1)**
- Volatility models: **GARCH, GJR-GARCH, EGARCH**
- Plus **GARCH-M** variants paired with each mean equation.

Selection Rule (Course-Aligned, High-Impact):

Because daily return predictability in the mean is typically weak (so RMSE differences are often tiny), we select the final model using a *diagnostics-first* approach:

1. **Model Adequacy Filter (Primary):** Keep only models whose **standardized residuals** pass:
   - Ljung–Box test: no remaining serial correlation (p-value > 0.05)
   - ARCH-LM test: no remaining ARCH effects (p-value > 0.05)
2. **Parsimony (Primary Tie-Breaker):** Among models that pass (1), choose the **lowest BIC**.
3. **Forecast sanity check (Secondary):** Report test-set **RMSE** of the fitted conditional mean as an additional check, but it is not the main driver.

```r
# Mean models
mean_grid <- list(
  "AR1"    = c(1, 0),
  "MA1"    = c(0, 1),
  "ARMA11" = c(1, 1)
)

# Vol models
vol_grid <- list(
  "GARCH"  = "sGARCH",    # standard GARCH(1,1)
  "GJR"    = "gjrGARCH",  # leverage/asymmetry
  "EGARCH" = "eGARCH"     # log-variance asymmetry
)

# Build candidate set
combos <- list()

# 9 base models
for (mname in names(mean_grid)) {
  for (vname in names(vol_grid)) {
    model_name <- paste0(mname, "-", vname)
    combos[[model_name]] <- list(
      arma    = mean_grid[[mname]],
      garch   = vol_grid[[vname]],
      archm   = FALSE,
      archpow = 1
    )
  }
}


for (mname in names(mean_grid)) {
  model_name <- paste0(mname, "-GARCHM")
  combos[[model_name]] <- list(
    arma    = mean_grid[[mname]],
    garch   = "sGARCH",
    archm   = TRUE,
    archpow = 1
  )
}


sse_train_vec <- as.numeric(sse_train)

safe_ugarchfit <- function(spec, data) {
  solvers <- c("hybrid", "solnp", "gosolnp")
  for (s in solvers) {
```

```r
    fit_try <- tryCatch(
      ugarchfit(
        spec = spec,
        data = data,
        solver = s,
        fit.control = list(stationarity = 1, fixed.se = 0, scale = 1),
        solver.control = list(trace = 0)
      ),
      error = function(e) e
    )
    if (!inherits(fit_try, "error")) return(fit_try)
  }
  return(NULL)
}

apply_starts <- function(spec, spec_def) {
  if (spec_def$garch == "sGARCH") {
    if (isTRUE(spec_def$archm)) {
      setstart(spec) <- list(omega = 0.1, alpha1 = 0.05,
                             beta1 = 0.90, archm = 0.0)
    } else {
      setstart(spec) <- list(omega = 0.1, alpha1 = 0.05, beta1 = 0.90)
    }
  } else if (spec_def$garch == "gjrGARCH") {
    setstart(spec) <- list(omega = 0.1, alpha1 = 0.05,
                           beta1 = 0.85, gamma1 = 0.05)
  } else if (spec_def$garch == "eGARCH") {
    setstart(spec) <- list(omega = -0.1, alpha1 = 0.10,
                           beta1 = 0.90, gamma1 = 0.05)
  }
  spec
}

results <- data.frame(
  Model     = character(),
  BIC       = numeric(),
  RMSE      = numeric(),
  LB_p      = numeric(),
  ARCH_p    = numeric(),
  PassDiag  = logical(),
  stringsAsFactors = FALSE
)

models_list <- list()
spec_list   <- list()

for (name in names(combos)) {
```

```r
spec_def <- combos[[name]]
cat("Fitting:", name, "\n")

spec <- ugarchspec(
  variance.model = list(model = spec_def$garch, garchOrder = c(1, 1)),
  mean.model     = list(
    armaOrder    = spec_def$arma,
    include.mean = TRUE,
    archm        = spec_def$archm,
    archpow      = spec_def$archpow
  ),
  distribution.model = "std"
)


fit <- safe_ugarchfit(spec, sse_train_vec)


if (is.null(fit)) {
  spec2 <- apply_starts(spec, spec_def)
  fit   <- safe_ugarchfit(spec2, sse_train_vec)
  spec  <- spec2
}

if (is.null(fit)) {
  cat("  -> FAILED (skipping):", name, "\n")
  next
}

models_list[[name]] <- fit
spec_list[[name]]   <- spec

spec_fixed <- spec
setfixed(spec_fixed) <- as.list(coef(fit))

filt <- tryCatch(
  ugarchfilter(spec_fixed, sse_xts),
  error = function(e) e
)
if (inherits(filt, "error")) {
  cat("  -> FILTER FAILED (skipping):", name, "\n")
  next
}

pred_all  <- fitted(filt)
pred_test <- pred_all[(n_train + 1):n_total]
real_test <- sse_test
```

```r
  err      <- as.numeric(real_test) - as.numeric(pred_test)
  rmse_val <- sqrt(mean(err^2, na.rm = TRUE))

  std_resid <- residuals(fit, standardize = TRUE)
  std_num   <- as.numeric(std_resid)

  # Ljung-Box: no remaining serial correlation
  lb_p <- tryCatch(
    Box.test(std_num, lag = 10, type = "Ljung-Box")$p.value,
    error = function(e) NA_real_
  )

  # ARCH-LM: no remaining ARCH
  arch_p <- tryCatch(
    ArchTest(std_num, lags = 12)$p.value,
    error = function(e) NA_real_
  )

  pass_diag <- (!is.na(lb_p) && !is.na(arch_p) && lb_p > 0.05 && arch_p > 0.05)

  # Store
  results <- rbind(
    results,
    data.frame(
      Model    = name,
      BIC      = infocriteria(fit)[2],
      RMSE     = rmse_val,
      LB_p     = lb_p,
      ARCH_p   = arch_p,
      PassDiag = pass_diag
    )
  )
}
```

```
Fitting: AR1-GARCH
Fitting: AR1-GJR
Fitting: AR1-EGARCH
Fitting: MA1-GARCH
Fitting: MA1-GJR
Fitting: MA1-EGARCH
Fitting: ARMA11-GARCH
Fitting: ARMA11-GJR
Fitting: ARMA11-EGARCH
Fitting: AR1-GARCHM
Fitting: MA1-GARCHM
Fitting: ARMA11-GARCHM
```

```r
if (nrow(results) < 2) {
  stop("Too few successful fits.
       Consider simplifying the model set or checking the data.")
}

results_display <- results %>%
  arrange(desc(PassDiag), BIC, RMSE)

kable(
  results_display,
  digits = 6,
  caption = " Results: Diagnostics-First Selection (Primary)
                            + BIC (Tie-Breaker) + RMSE (Secondary)"
)
```

Table 2: Results: Diagnostics-First Selection (Primary) + BIC (Tie-Breaker) + RMSE (Secondary)

	Model	BIC	RMSE	LB_p	ARCH_p	PassDiag
Chi-squared6	ARMA11-GARCH	3.324688	1.027090	0.106110	0.295367	TRUE
Chi-squared8	ARMA11-EGARCH	3.325507	1.026819	0.093888	0.194550	TRUE
Chi-squared7	ARMA11-GJR	3.326921	1.027133	0.093997	0.344915	TRUE
Chi-squared11	ARMA11-GARCHM	3.327311	1.026879	0.100343	0.298721	TRUE
Chi-squared3	MA1-GARCH	3.323135	1.025127	0.028110	0.309865	FALSE
Chi-squared	AR1-GARCH	3.323135	1.025126	0.028163	0.309841	FALSE
Chi-squared2	AR1-EGARCH	3.323807	1.025203	0.025364	0.212659	FALSE
Chi-squared5	MA1-EGARCH	3.323807	1.025204	0.025346	0.212649	FALSE
Chi-squared1	AR1-GJR	3.325408	1.025048	0.027072	0.357893	FALSE
Chi-squared4	MA1-GJR	3.325408	1.025051	0.026982	0.357829	FALSE
Chi-squared9	AR1-GARCHM	3.325767	1.024937	0.026776	0.312500	FALSE
Chi-squared10	MA1-GARCHM	3.325767	1.024939	0.026722	0.312497	FALSE

```r
# Select winner:

passed <- results %>% filter(PassDiag)

if (nrow(passed) >= 1) {
  passed <- passed %>% arrange(BIC, RMSE)
  best_model_name <- passed$Model[1]
  selection_note <- "Winner chosen as the lowest-BIC model
                      among those passing Ljung-Box and ARCH-LM diagnostics."
} else {
```

```r
  results_fallback <- results %>% arrange(BIC, RMSE)
  best_model_name <- results_fallback$Model[1]
  selection_note <- "No model passed both diagnostics; winner
                      chosen by lowest BIC overall (with limitations noted)."
}

best_fit_obj <- models_list[[best_model_name]]
best_spec    <- spec_list[[best_model_name]]

cat("\nSelected Model:", best_model_name, "\n")
```

```
Selected Model: ARMA11-GARCH
```

```r
cat("Selection rule:", selection_note, "\n")
```

```
Selection rule: Winner chosen as the lowest-BIC model
                      among those passing Ljung-Box and ARCH-LM diagnostics.
```

5.1 Final Parameter Estimation (QMLE)

We present the coefficients of the **ARMA11-GARCH** model. These were estimated jointly (Mean + Variance) using a Student-t likelihood.

Note: Robust (Bollerslev–Wooldridge) standard errors correspond to QMLE-type inference.

```r
show(best_fit_obj)
```

```
*---------------------------------*
*          GARCH Model Fit        *
*---------------------------------*

Conditional Variance Dynamics
-----------------------------------
GARCH Model : sGARCH(1,1)
Mean Model  : ARFIMA(1,0,1)
Distribution    : std

Optimal Parameters
------------------------------------
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.040076    0.017915   2.2370 0.025284
ar1    -0.891058    0.070185 -12.6959 0.000000
ma1     0.905473    0.065303  13.8657 0.000000
```

```
omega   0.007994    0.003168   2.5238 0.011608
alpha1  0.054349    0.007608   7.1440 0.000000
beta1   0.944651    0.007106 132.9356 0.000000
shape   4.571104    0.388373  11.7699 0.000000

Robust Standard Errors:
        Estimate  Std. Error  t value Pr(>|t|)
mu      0.040076    0.017489   2.2914 0.021938
ar1    -0.891058    0.046242 -19.2694 0.000000
ma1     0.905473    0.042972  21.0714 0.000000
omega   0.007994    0.003647   2.1923 0.028361
alpha1  0.054349    0.008222   6.6101 0.000000
beta1   0.944651    0.007496 126.0129 0.000000
shape   4.571104    0.368542  12.4032 0.000000

LogLikelihood : -4998.878

Information Criteria
------------------------------------

Akaike       3.3108
Bayes        3.3247
Shibata      3.3108
Hannan-Quinn 3.3158

Weighted Ljung-Box Test on Standardized Residuals
------------------------------------
                        statistic  p-value
Lag[1]                      1.308 0.252780
Lag[2*(p+q)+(p+q)-1][5]     4.767 0.007186
Lag[4*(p+q)+(p+q)-1][9]     8.127 0.051791
d.o.f=2
H0 : No serial correlation

Weighted Ljung-Box Test on Standardized Squared Residuals
------------------------------------
                        statistic p-value
Lag[1]                      3.347 0.06733
Lag[2*(p+q)+(p+q)-1][5]     5.946 0.09261
Lag[4*(p+q)+(p+q)-1][9]     7.500 0.16081
d.o.f=2

Weighted ARCH LM Tests
------------------------------------
            Statistic Shape Scale P-Value
ARCH Lag[3]     4.074 0.500 2.000 0.04355
ARCH Lag[5]     4.406 1.440 1.667 0.14068
ARCH Lag[7]     4.961 2.315 1.543 0.22845
```

```
Nyblom stability test
------------------------------------
Joint Statistic:  1.9434
Individual Statistics:
mu     0.11510
ar1    0.04076
ma1    0.04206
omega  0.50787
alpha1 0.96461
beta1  1.14941
shape  0.88560

Asymptotic Critical Values (10% 5% 1%)
Joint Statistic:          1.69 1.9 2.35
Individual Statistic:     0.35 0.47 0.75

Sign Bias Test
------------------------------------
                   t-value   prob sig
Sign Bias            1.040 0.2983
Negative Sign Bias   1.455 0.1459
Positive Sign Bias   1.098 0.2725
Joint Effect         3.371 0.3378


Adjusted Pearson Goodness-of-Fit Test:
------------------------------------
  group statistic p-value(g-1)
1    20     43.58     0.001079
2    30     54.59     0.002764
3    40     71.77     0.001076
4    50     80.83     0.002830


Elapsed time : 0.3464499
```

5.2 Residual Analysis, ARCH Test and Normality (Final Model)

We confirm the adequacy of the **selected** model by examining the **Standardized Residuals** ($z_t = \epsilon_t/\sigma_t$).

```r
std_resid <- residuals(best_fit_obj, standardize = TRUE)
std_resid_num <- as.numeric(std_resid)


lb_test <- Box.test(std_resid_num, lag = 10, type = "Ljung-Box")
```

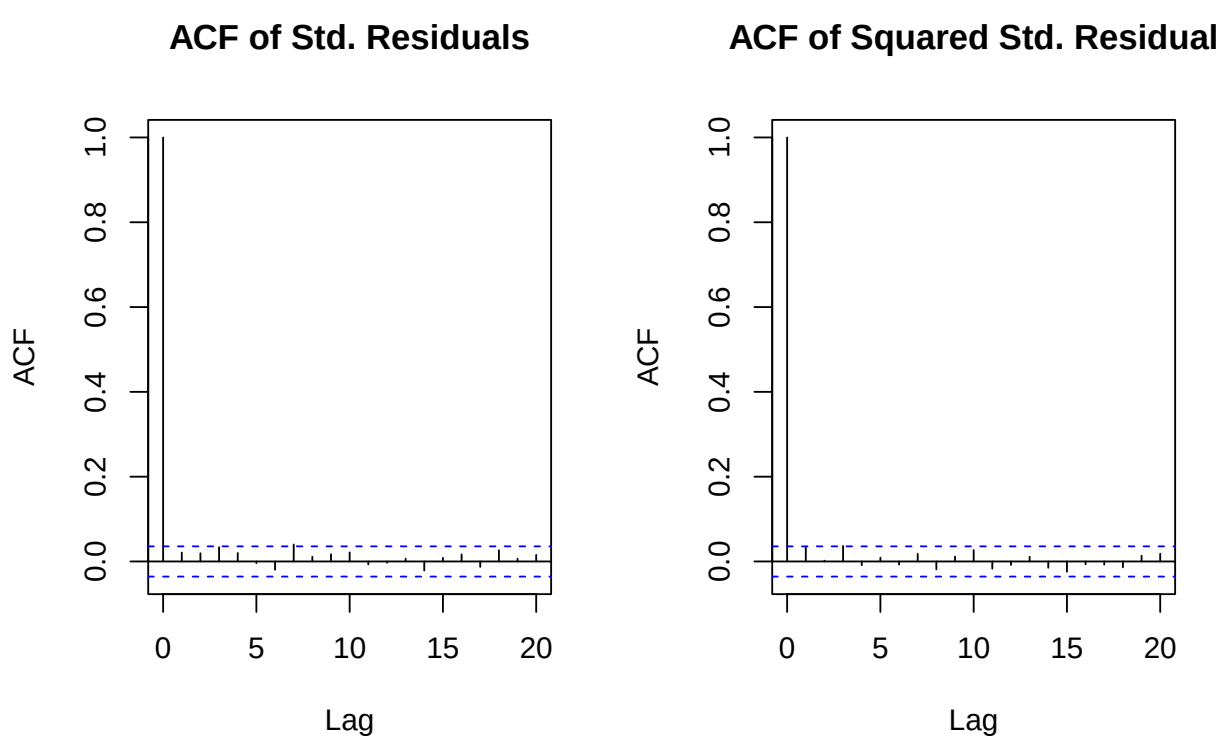
```r
arch_test <- ArchTest(std_resid_num, lags = 12)

print(lb_test)
```

```
    Box-Ljung test

data:  std_resid_num
X-squared = 15.78, df = 10, p-value = 0.1061
```

```r
print(arch_test)
```

```
    ARCH LM-test; Null hypothesis: no ARCH effects

data:  std_resid_num
Chi-squared = 14.084, df = 12, p-value = 0.2954
```

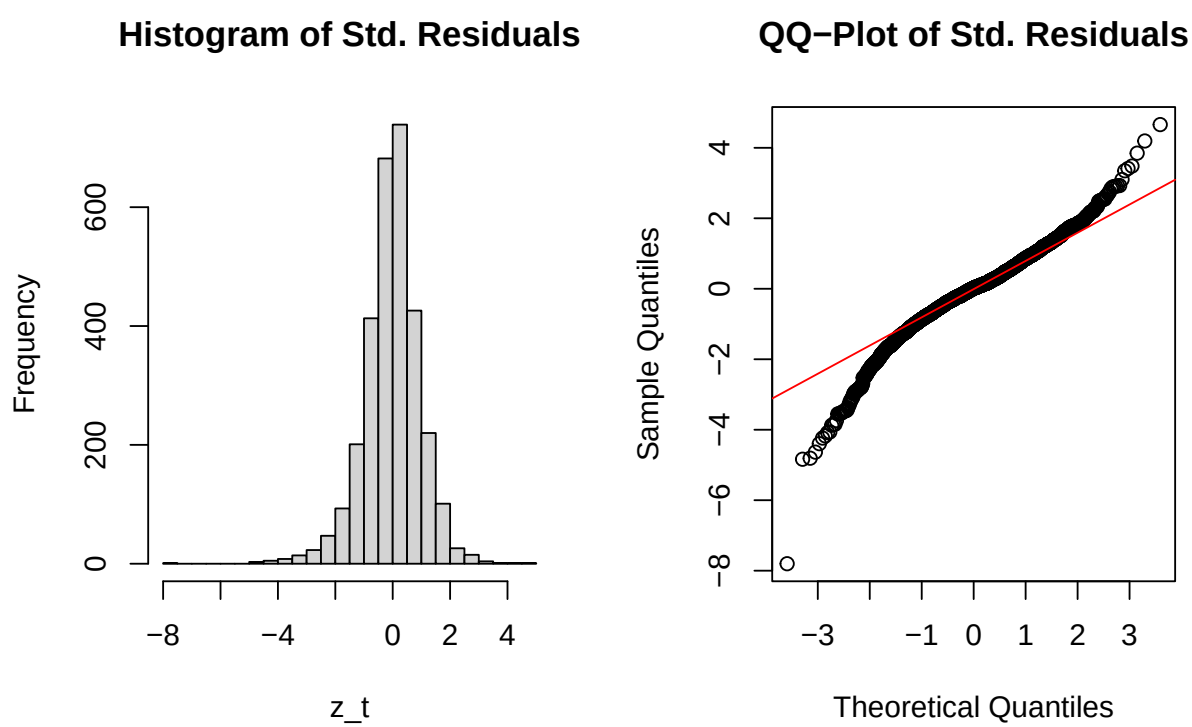
```r
par(mfrow = c(1, 2))
acf(std_resid_num,       main = "ACF of Std. Residuals",        lag.max = 20)
acf(std_resid_num^2,     main = "ACF of Squared Std. Residual", lag.max = 20)
```



```r
par(mfrow = c(1, 1))

# Normality checks
jb_test <- tseries::jarque.bera.test(std_resid_num)
print(jb_test)
```

```
    Jarque Bera Test

data:  std_resid_num
X-squared = 1446.7, df = 2, p-value < 2.2e-16
```

```r
par(mfrow = c(1, 2))
hist(std_resid_num, breaks = 30,
     main = "Histogram of Std. Residuals", xlab = "z_t")
qqnorm(std_resid_num, main = "QQ-Plot of Std. Residuals")
qqline(std_resid_num, col = "red")
```




```r
par(mfrow = c(1, 1))
```

6. Dynamic Forecasting (Investment Application)

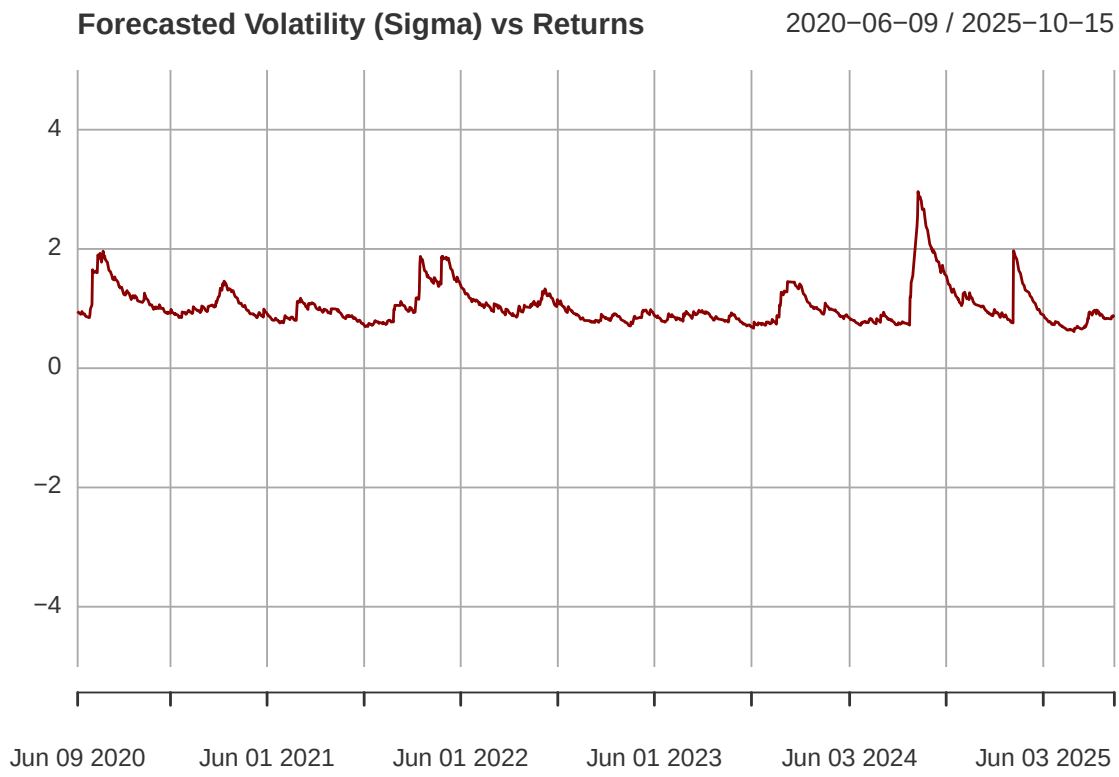
We perform a final **rolling forecast** (re-estimating parameters approximately once per year) to generate a dynamic volatility signal.

```
winner_def <- combos[[best_model_name]]

final_spec <- ugarchspec(
  variance.model = list(model = winner_def$garch, garchOrder = c(1, 1)),
  mean.model     = list(
    armaOrder    = winner_def$arma,
    include.mean = TRUE,
    archm        = winner_def$archm,
    archpow      = winner_def$archpow
  ),
  distribution.model = "std"
)

roll_forecast <- ugarchroll(
  spec         = final_spec,
  data         = sse_xts,
  n.ahead      = 1,
  n.start      = n_train,
  refit.every  = 250,
  refit.window = "recursive"
)

forecast_df <- as.data.frame(roll_forecast)
```

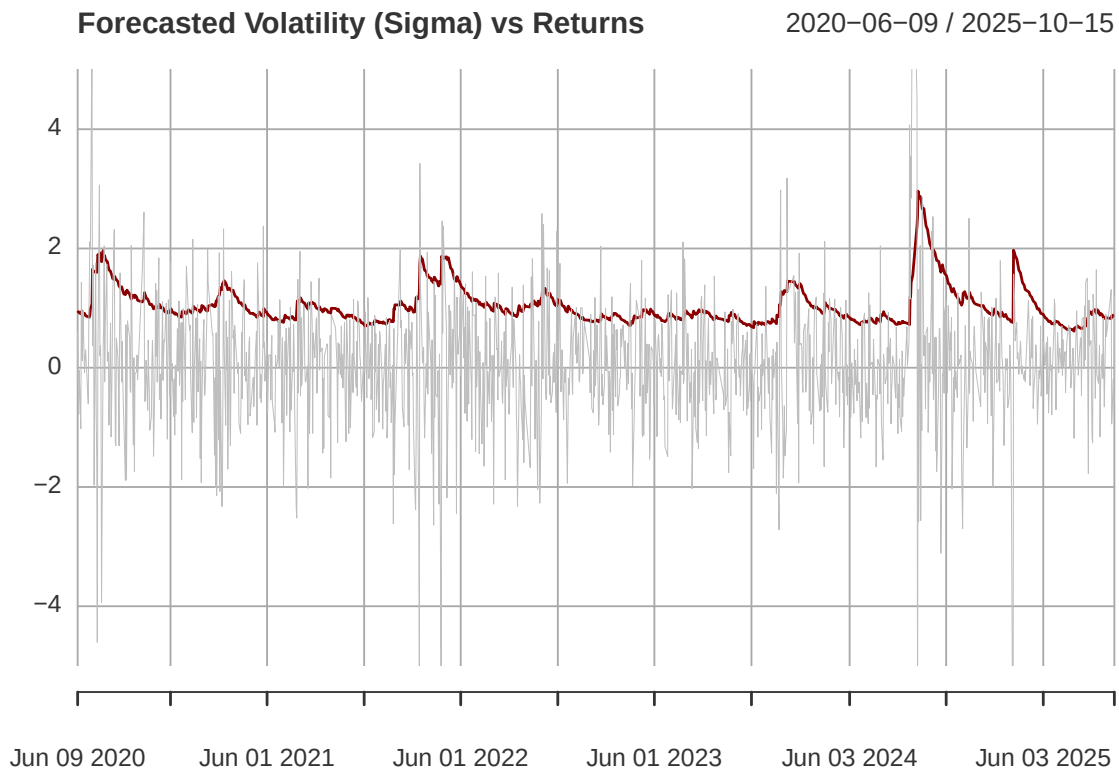
6.1 Performance Review

```
dates_test <- index(sse_xts)[(n_train + 1):n_total]
if (length(forecast_df$Sigma) != length(dates_test)) {
  dates_test <- tail(dates_test, length(forecast_df$Sigma))
}

sigma_xts <- xts(forecast_df$Sigma, order.by = dates_test)

par(mar = c(3, 4, 3, 1))
plot(
  sigma_xts,
  main = "Forecasted Volatility (Sigma) vs Returns",
  col  = "darkred",
  lwd  = 1.5,
  ylim = c(-5, 5),
```

```
  yaxis.right = FALSE
)
```




```
lines(sse_test, col = "gray", lwd = 0.5)
legend(
  "topleft",
  legend = c("Predicted Risk (Sigma)", "Actual Returns"),
  col    = c("darkred", "gray"),
  lty    = 1,
  cex    = 0.8,
  bty    = "n"
)
```



7. Executive Summary

Based on the rigorous Train–Test validation:

1. **Stationarity:** ADF testing on the training returns rejects a unit root, so returns are treated as stationary.

2. **ARCH Confirmation:** A baseline AR(1) mean model exhibits significant ARCH effects in residuals (ARCH-LM and autocorrelation in squared residuals), motivating GARCH-type volatility modelling.

3. **Candidate Set (Course Scope):** Mean candidates {AR(1), MA(1), ARMA(1,1)} are selected by training-set correlograms. Volatility candidates are restricted to GARCH, GJR-GARCH, EGARCH, and GARCH-in-Mean as standard extensions covered in class and commonly used in equity index volatility modelling.

4. **Model Selection (Diagnostics-First):** We estimate an assessment of 12 joint Mean–Variance models. We first retain only models that pass checks on standardized residuals (Ljung–Box p-value > 0.05 and ARCH-LM p-value > 0.05). Among those, we select the model with the lowest BIC (parsimony). Test-set RMSE of the conditional mean is reported as a secondary forecast sanity check rather than a primary selection criteria.

5. **Final Residual Analysis:** The selected model's standardized residuals show no material serial correlation and no remaining ARCH effects; squared-residual correlograms do not indicate additional volatility clustering.

6. **Robustness:** Normality checks on standardized residuals typically reject strict Normality; we appropriately use QML estimation to estimatie the regression parameters.

7. **Forecasting Application:** Rolling 1-step-ahead volatility forecasts (with periodic refits) provide a time-varying risk signal suitable for volatility targeting or risk-based position sizing in an SPFTZ-linked portfolio.

Limitations

Despite the comprehensive analysis, several limitations remain:

- **Residual mean dynamics.**
  The ARMA(1,1) mean equation eliminates most, but not all, serial dependence in returns. We accept this because daily equity index returns are empirically hard to forecast in mean, and our primary objective is accurate **volatility modelling** rather than market timing.

- **Structural breaks and regimes.**
  The sample covers multiple crises and policy regimes (e.g. GFC, Covid-19). A single GARCH specification cannot fully capture all such shifts. In practice, a better approach might be to reestimate model parameters using a rolling window approach, capturing changing market dynamics in different regimes.

- **Data quality and extreme observations.**
  The dataset contains a few very large single-day moves. The Student-t GARCH framework is robust to such outliers, but they still influence parameter estimates. In a production environment, we would investigate these episodes individually and consider robust estimation or data cleaning rules.

From a business perspective, the key message is that the SSE Composite has **very small average daily drift** but **meaningful and persistent risk**. Typical daily changes are around **±1.6 units on our 100×log-return scale** (roughly ±1½%), but the worst days can reach **about ±9%**, especially during crisis episodes. This means that, while the index is not strongly predictable in direction, **the size of daily moves is highly variable and must be actively managed** using a dynamic volatility estimate rather than a fixed "one number" for risk.